Advanced Algorithms Assignment 2 Report

1. Problem and Algorithms

Problem
Given a connected, undirected graph with non-negative integer edge weights, the objective is to separate the vertices into two non-empty disjoint sets such that the total weight of the edges crossing between them is minimised. This is the mathematical model for finding the cheapest or most vulnerable bottleneck required to break a network into two isolated components.

Stoer-Wagner
Stoer-Wagner is deterministic: one run always returns the minimum cut. It runs n − 1 phase. Each phase grows a set A, repeatedly adding the vertex most tightly connected to A (maximum adjacency ordering). The last vertex added, t, cut off from the rest, is a minimum cut between t and the vertex s added just before it: the “cut of the phase”. The phase records this cut, then merges s and t. The lightest recorded cut is the global minimum, because every minimum cut either separates the s and t of some phase or survives their merge.

My implementation uses an n × n adjacency matrix, so each phase costs O(n²) and the whole algorithm O(n³), regardless of m. A heap-based version would run in O(nm + n² log n). Unlike Karger and Karger-Stein, it never guesses: it pays more per run but needs only one.

Karger
Karger is a randomised algorithm that runs counterintuitively to other algorithms where instead of trying to find the minimum cut directly, the algorithm randomly destroys the graph until there are only two super vertices remaining. When running the algorithm, it picks a random edge and contracts its endpoints into a single vertex. To visualize this, think of taking two connected vertices and fusing them into a super-vertex, reducing the total number of vertices to n - 1. Any resulting parallel edges are kept or combined, while self-loops which are edges where both endpoints connect to each other are discarded. This is done in every step and because the minimum cut usually has very few edges in relation to the total edge count, this random contraction process is repeated so much that the lowest weight cut will become a candidate for the global minimum cut because this process tends to avoid cutting the min-cut rather than hitting it.

The Karger’s single run has a time complexity of $O(m\log n)$ because I used the edge-ordering Union-Find approach in my implementation. Because the algorithm is a randomised one, its success probability bound on a single run is $\frac{2}{n(n-1)}$ and has to be repeated $O(n^2\log n)$ times if you want the success probability to reach 99%, thus the total time complexity would look something like $O(mn^2\log^2 n)$.

The key difference in what makes Karger different from Karger-Stein and Stoer-Wagner boils down to the core concept of the algorithm which is the contractions. Stoer-Wagner is a deterministic algorithm that guarantees success on a single run and does not use contractions unlike Karger which uses randomised contractions and requires multiple runs to obtain a decent success rate. Karger-Stein differs from Karger because it uses divide and conquer by contracting the graph into a smaller size and uses recursive calls to increase the speed and success rate.

Karger-Stein
Karger-Stein is an optimisation of the standard Karger algorithm that reduces the number of repetitions needed by using a recursive divide and conquer strategy. It is based on the idea that random edge contractions are very safe when a graph is large, but risky when the graph shrinks down to just a few vertices. The Karger-Stein algorithm contracts the graph $(1 + n/\sqrt{2})$ until vertices remain, where then it will branch into two recursive calls on the graph and go through them in parallel to find the minimum cut.

The time complexity of a single run of Karger-Stein is $O(n^2 \log n)$, with a success probability bound of $O(1/\log n)$. To reach a 99% confidence interval, the algorithm is repeated $O(\log^2 n)$ times, which has a total time complexity of $O(n^2 \log^3 n)$, which is an improvement over the standard Karger algorithm's total time. The difference of Karger-Stein from the others is its recursion tree. Karger-Stein has a dense branched tree of sub-problems instead of matrix scans or individual trials.

2. Theoretical Comparison and Predictions

Refer to predictions/name.md (name being either brian, kartikay, ethan)

3. Experimental Methodology

Our question is when a deterministic algorithm defeats a randomized one to find the global minimum cut, and why. All numbers cited come from just one run of the benchmarks; therefore, we will describe this run in sufficient detail to reproduce it completely: details about what was implemented, the specific graphs run against, how the correctness was verified, how times and successes were measured, and where measurements were weakest.

Algorithm implementations under test
Each of these three algorithms implements exactly one interface written in C++, which includes a seed and an edge list with positive integers for weights. These programs all return either the total weight of the cut that they have found along with one side of the cut.
It appears that how each of these programs store their respective graphs explains most of the results. Therefore, we include this information here.

- **Stoer-Wagner** (deterministic) uses an n × n adjacency matrix and n − 1 maximum-adjacency phases, so it does $\Theta(n^3)$ work whatever the number of edges.
- **Karger** (randomised) performs one contraction run per call: it sorts the edges by random keys $-\ln(U)/w$ and merges endpoints with union-find, at a cost of $O(m \log m)$.
- **Karger-Stein** (randomised) contracts to $t = \lceil 1 + n/\sqrt{2} \rceil$ vertices twice, compacts each result, recurses on both and keeps the lighter cut.

The harness does all the repeating, so the cost of one run and its chance of success are measured separately.

Both Karger and Karger-Stein make use of only a single contraction pass through the original graph. Since there is a separate contract-and-recursion loop for each call to Karger or Karger-Stein, we can measure the probability of a successful outcome of each individual contraction pass (i.e., finding a valid light cut) and the cost of each contraction pass separately and combine those results later.

Input Graph Families

For our experiments we chose five families of graphs such that we could evaluate different properties of these three algorithms: high densities of edges, a large number of edges that would create only one very weak link among all edges in the graph, a large number of cuts having almost equal minimum weights, and several cuts having values that are slightly larger than the minimum value. Table 1 lists the five families.

Family	Construction	Edges at n = 100	Minimum cut	Why it is included
random_dense	Each pair joined with probability 0.5, weights uniform in 1 to 100	about 2,460	The lightest vertex (1,636 to 1,822)	Dense graphs with no planted structure
random_sparse	As above with probability 0.1, plus edges to connect any separate components	about 485	Usually a light vertex (132 to 147 at n = 100)	Sparse graphs, where m is far below n^2
barbell	Two cliques of n/2 vertices, weights 1 to 100, joined by one edge of weight 1	2,451	1, the bridge	One weak link hidden among many heavy edges
cycle	A single cycle of unit edges	100	2, achieved by n(n − 1)/2 different cuts	The textbook case where Karger's bound is tight for any one cut
many_cuts	A ring of unit-weight cliques of about 7 vertices, joined by bridges of weight 3 and one weak bridge of weight 2	322	5, the weak bridge plus any strong one; many cuts of 6 close behind	Many near-minimum cuts, designed to be hard for random contraction

To evaluate the performance of each family we tested three sizes of n = 25, 50, and 100, with

three instances per size. Every instance has been created using a unique seed that is determined from the family name, n and the instance number (using FNV-1a and splitmix64) for every algorithm. Therefore every algorithm will see the exact same graphs in every run, on any machine. Only two of our families did not utilize the seed to generate the family structure: cycle and many_cuts produced the same graph for all three instances, and barbell only changes the clique weights. Thus the three instances of these families are repeated and not independently sampled, which explains why their success rates in Table 3 have agreed up to the last digit.

Evaluation of ground-truth and evaluation of correctness

Stoer-Wagner's answer is taken as the true minimum, so we checked it first: it matches an exhaustive brute force on nine hand-checked test graphs (run under AddressSanitizer and UBSan) and on 3,000 random graphs with up to 16 vertices. During the benchmark, the harness also recomputes the weight of every returned cut and stops if a randomised result is ever lighter than the truth. Neither check failed in 9,450 runs.

Measurement

The time for each algorithm is recorded using std::chrono::steady_clock only over the time period of calling an algorithm. The creation of the graph is done outside of the timing region; however, all conversions performed by each algorithm for converting the list of edges to its own internal representation (matrix, sorted key list, union-find) are included in the timing region because they are considered part of what each algorithm costs. Prior to taking measurements, we keep the CPU active for 0.5 seconds before taking measurements since without this activity, the very smallest initial instances ran approximately 3x faster when the CPU increased its clock speed. Each measured set begins with one un-timed warm-up call.

Stoer-Wagner was called ten times per instance. Karger and Karger-Stein were called 100 times per instance with seeds 0 thru 99; each call records both its time taken and whether it successfully identified the minimum cut. When combined with three instances each, there are thirty calls per (family,n) for Stoer-Wagner and 300 runs per (family, n, algorithm) for the randomized algorithms. The results of each run are stored in a separate csv row (results/trial.csv; 9,450 total rows), and all statistics presented below are calculated from this data file by analysis/plots.py.

Derived quantities

Since a run can be interrupted by the OS, we report median times. The success probability p of one run is defined as the proportion of successful identification of the minimum cut out of the 300 runs (with a 95% Wilson interval); from p we calculate the number of runs k required to identify the minimum cut with 99% confidence ($k = \lceil \ln 0.01 / \ln(1-p) \rceil$) and the time required to reach 99% confidence as k * mean run time. Since k * mean run time represents the cost of performing k runs as their sum, we chose to multiply by the mean rather than median.

Environment

We used 1 computer, Brian's Mac to run the results, which had the specs below:
Macbook Pro M4
Chipset: M4 (10-core CPU / 10-core GPU)
RAM: 16gb
OS: macOS Sequoia (Version 15.8.1)

Limitations

Four limitations affect how far the conclusions reach. First, the largest graphs have 100 vertices, so every growth rate below rests on two doublings, and any crossover beyond n = 100 is an extrapolation. Second, 300 runs pin down large success probabilities well but small ones loosely: on random_dense at n = 100 Karger succeeded in 5 of 300 runs, so p is somewhere between 0.7% and 3.8%, and the number of runs needed for 99% is anywhere from about 120 to 650. Third, cycle, many_cuts and (in structure) barbell give three identical instances, so their results describe one graph each. Fourth, the explanation we give for Stoer-Wagner's dependence on graph structure rests on counting a proxy in a separate, untimed copy of its loops, not on hardware counters.

4. Results and Analysis

Input Family	Algorithm	n	Mean Time (ms)	Std Time (ms)	Success Rate	Mean Found Cut	True Min Cut	Total Runs
Barbell	Karger	25	0.0056	0.0005	98.3%	15.55	1.00	300
	Karger	50	0.0234	0.0012	100.0%	1.00	1.00	300
	Karger	100	0.1093	0.0102	100.0%	1.00	1.00	300
	Karger-Stein	25	4.2320	0.0534	100.0%	1.00	1.00	300
	Karger-Stein	50	17.3531	0.3055	100.0%	1.00	1.00	300

	Karger-Stein	100	69.5642	0.9745	100.0%	1.00	1.00	300
	Stoer-Wagner	25	0.0101	0.0017	100.0%	1.00	1.00	30
	Stoer-Wagner	50	0.0701	0.0062	100.0%	1.00	1.00	30
	Stoer-Wagner	100	0.6453	0.0361	100.0%	1.00	1.00	30
Cycle	Karger	25	0.0016	0.0002	100.0%	2.00	2.00	300
	Karger	50	0.0026	0.0003	100.0%	2.00	2.00	300
	Karger	100	0.0049	0.0011	100.0%	2.00	2.00	300
	Karger-Stein	25	4.2428	0.0657	100.0%	2.00	2.00	300
	Karger-Stein	50	17.3873	0.1530	100.0%	2.00	2.00	300

	Karger-Stein	100	69.6752	0.7405	100.0%	2.00	2.00	300
	Stoer-Wagner	25	0.0093	0.0011	100.0%	2.00	2.00	30
	Stoer-Wagner	50	0.0621	0.0027	100.0%	2.00	2.00	30
	Stoer-Wagner	100	0.4211	0.0028	100.0%	2.00	2.00	30
Many Cuts	Karger	25	0.0038	0.0004	15.0%	8.01	5.00	300
	Karger	50	0.0066	0.0003	14.0%	7.24	5.00	300
	Karger	100	0.0140	0.0034	10.0%	6.60	5.00	300
	Karger-Stein	25	4.6733	0.1164	98.0%	5.03	5.00	300
	Karger-Stein	50	17.8780	0.3031	97.0%	5.03	5.00	300

	Karger-Stein	100	69.3095	0.8143	97.0%	5.03	5.00	300
	Stoer-Wagner	25	0.0104	0.0017	100.0%	5.00	5.00	30
	Stoer-Wagner	50	0.0668	0.0041	100.0%	5.00	5.00	30
	Stoer-Wagner	100	0.4749	0.0063	100.0%	5.00	5.00	30
Random Dense	Karger	25	0.0054	0.0007	15.7%	533.31	296.00	300
	Karger	50	0.0236	0.0023	13.0%	1115.33	686.67	300
	Karger	100	0.1040	0.0038	1.7%	2464.43	1710.00	300
	Karger-Stein	25	4.8457	0.0998	93.7%	300.59	296.00	300
	Karger-Stein	50	19.8232	0.9294	90.3%	696.00	686.67	300

	Karger-Stein	100	82.3101	4.8691	85.7%	1720.57	1710.00	300
	Stoer-Wagner	25	0.0105	0.0029	100.0%	296.00	296.00	30
	Stoer-Wagner	50	0.0799	0.0167	100.0%	686.67	686.67	30
	Stoer-Wagner	100	0.9417	0.0256	100.0%	1710.00	1710.00	30
Random Sparse	Karger	25	0.0018	0.0003	54.3%	19.61	10.00	300
	Karger	50	0.0055	0.0007	41.7%	65.71	36.33	300
	Karger	100	0.0191	0.0015	13.3%	295.10	141.67	300
	Karger-Stein	25	3.9137	0.2202	99.7%	10.01	10.00	300
	Karger-Stein	50	15.7706	0.6921	100.0%	36.33	36.33	300

	Karger-Stein	100	64.8983	1.7352	97.7%	142.75	141.67	300
	Stoer-Wagner	25	0.0102	0.0017	100.0%	10.00	10.00	30
	Stoer-Wagner	50	0.0722	0.0093	100.0%	36.33	36.33	30
	Stoer-Wagner	100	0.8722	0.0269	100.0%	141.67	141.67	30

```
briantran@Brians-MacBook-Pro-2 mincut-comparison % make test
c++ -std=c++20 -Wall -Wextra -Iinclude -Icommon -g -O1 -fsanitize=address,undefined tests/tests.cpp stoer_wagner/stoer_wagner.cpp karger/karger.cpp karger_stein/karger_stein.cpp co
mmon/brute_force.cpp common/generators.cpp -o build/tests
./build/tests
== brute force
  PASS  single edge
  PASS  weighted path
  PASS  cycle of 5
  PASS  K4
  PASS  two triangles + bridge
  PASS  parallel edges
  PASS  Stoer-Wagner paper
  PASS  cycle
  PASS  barbell
== Stoer-Wagner
  PASS  single edge
  PASS  weighted path
  PASS  cycle of 5
  PASS  K4
  PASS  two triangles + bridge
  PASS  parallel edges
  PASS  Stoer-Wagner paper
  PASS  cycle
  PASS  barbell
== Karger
  PASS  single edge              found minimum in 20/20 seeds
  PASS  weighted path            found minimum in 12/20 seeds
  PASS  cycle of 5               found minimum in 20/20 seeds
  PASS  K4                       found minimum in 14/20 seeds
  PASS  two triangles + bridge   found minimum in 6/20 seeds
  PASS  parallel edges           found minimum in 11/20 seeds
  PASS  Stoer-Wagner paper       found minimum in 9/20 seeds
  PASS  cycle                    found minimum in 20/20 seeds
  PASS  barbell                  found minimum in 10/20 seeds
== Karger-Stein
  PASS  single edge              found minimum in 20/20 seeds
  PASS  weighted path            found minimum in 19/20 seeds
  PASS  cycle of 5               found minimum in 20/20 seeds
  PASS  K4                       found minimum in 20/20 seeds
  PASS  two triangles + bridge   found minimum in 20/20 seeds
  PASS  parallel edges           found minimum in 13/20 seeds
  PASS  Stoer-Wagner paper       found minimum in 20/20 seeds
  PASS  cycle                    found minimum in 20/20 seeds
  PASS  barbell                  found minimum in 20/20 seeds

  OK (0 failures)
```

```
briantran@Brians-MacBook-Pro-2 mincut-comparison % make bench
c++ -std=c++20 -Wall -Wextra -Iinclude -Icommon -O2 -DNDEBUG bench/bench.cpp stoer_wagner/stoer_wagner.cpp karger/karger.cpp karger_stein/karger_stein.cpp common/brute_force.cpp co
mmon/generators.cpp -o build/bench
briantran@Brians-MacBook-Pro-2 mincut-comparison % ./build/bench --sizes 25,50,100 --instances 3 --runs 100 --reps 10 --out results/trial.csv
compiler: Apple LLVM 17.0.0 (clang-1700.0.13.5)
command: ./build/bench --sizes 25,50,100 --instances 3 --runs 100 --reps 10 --out results/trial.csv
writing:  results/trial.csv

family              n       m inst  algorithm      runs   median_ms       p    runs_99    time_99_ms
random_dense       25     140    0  stoer-wagner     10       0.0092   1.000          1         0.011
random_dense       25     140    0  karger          100       0.0049   0.120         37         0.187
random_dense       25     140    0  karger-stein    100       4.8793   0.950          2         9.728
random_dense       25     147    1  stoer-wagner     10       0.0092   1.000          1         0.010
random_dense       25     147    1  karger          100       0.0054   0.150         29         0.161
random_dense       25     147    1  karger-stein    100       4.8180   0.930          2         9.618
random_dense       25     151    2  stoer-wagner     10       0.0100   1.000          1         0.011
random_dense       25     151    2  karger          100       0.0054   0.200         21         0.118
random_dense       25     151    2  karger-stein    100       4.8698   0.930          2         9.728
random_dense       50     621    0  stoer-wagner     10       0.0691   1.000          1         0.077
random_dense       50     621    0  karger          100       0.0226   0.160         27         0.624
random_dense       50     621    0  karger-stein    100      19.5176   0.920          2        39.334
random_dense       50     648    1  stoer-wagner     10       0.0796   1.000          1         0.082
random_dense       50     648    1  karger          100       0.0239   0.100         44         1.088
random_dense       50     648    1  karger-stein    100      19.9348   0.880          3        59.778
random_dense       50     602    2  stoer-wagner     10       0.0772   1.000          1         0.082
random_dense       50     602    2  karger          100       0.0222   0.130         34         0.779
random_dense       50     602    2  karger-stein    100      19.8797   0.910          2        39.753
random_dense      100    2458    0  stoer-wagner     10       0.9466   1.000          1         0.947
random_dense      100    2458    0  karger          100       0.1029   0.020        228        23.677
random_dense      100    2458    0  karger-stein    100      81.0243   0.820          3       252.321
random_dense      100    2436    1  stoer-wagner     10       0.9464   1.000          1         0.943
random_dense      100    2436    1  karger          100       0.1019   0.030        152        15.730
random_dense      100    2436    1  karger-stein    100      80.9652   0.900          2       162.868
random_dense      100    2488    2  stoer-wagner     10       0.9360   1.000          1         0.935
random_dense      100    2488    2  karger          100       0.1042   0.000      never             -
random_dense      100    2488    2  karger-stein    100      81.3464   0.850          3       244.168
```

random_sparse	25	29	0	stoer-wagner	10	0.0103	1.000	1	0.011
random_sparse	25	29	0	karger	100	0.0015	0.430	9	0.015
random_sparse	25	29	0	karger-stein	100	3.8666	0.990	1	3.910
random_sparse	25	27	1	stoer-wagner	10	0.0098	1.000	1	0.010
random_sparse	25	27	1	karger	100	0.0016	0.550	6	0.010
random_sparse	25	27	1	karger-stein	100	3.8095	1.000	1	3.828
random_sparse	25	37	2	stoer-wagner	10	0.0089	1.000	1	0.009
random_sparse	25	37	2	karger	100	0.0018	0.650	5	0.010
random_sparse	25	37	2	karger-stein	100	3.9919	1.000	1	4.004
random_sparse	50	126	0	stoer-wagner	10	0.0776	1.000	1	0.074
random_sparse	50	126	0	karger	100	0.0049	0.260	16	0.082
random_sparse	50	126	0	karger-stein	100	15.8560	1.000	1	16.018
random_sparse	50	136	1	stoer-wagner	10	0.0686	1.000	1	0.071
random_sparse	50	136	1	karger	100	0.0056	0.140	31	0.175
random_sparse	50	136	1	karger-stein	100	15.5423	1.000	1	15.608
random_sparse	50	139	2	stoer-wagner	10	0.0728	1.000	1	0.071
random_sparse	50	139	2	karger	100	0.0057	0.850	3	0.017
random_sparse	50	139	2	karger-stein	100	15.6293	1.000	1	15.685
random_sparse	100	483	0	stoer-wagner	10	0.8747	1.000	1	0.878
random_sparse	100	483	0	karger	100	0.0197	0.140	31	0.602
random_sparse	100	483	0	karger-stein	100	64.5223	0.980	2	129.263
random_sparse	100	485	1	stoer-wagner	10	0.8784	1.000	1	0.883
random_sparse	100	485	1	karger	100	0.0181	0.180	24	0.438
random_sparse	100	485	1	karger-stein	100	64.6250	0.970	2	130.085
random_sparse	100	488	2	stoer-wagner	10	0.8448	1.000	1	0.855
random_sparse	100	488	2	karger	100	0.0197	0.080	56	1.097
random_sparse	100	488	2	karger-stein	100	64.7341	0.980	2	130.041
barbell	25	145	0	stoer-wagner	10	0.0102	1.000	1	0.010
barbell	25	145	0	karger	100	0.0055	0.980	2	0.011
barbell	25	145	0	karger-stein	100	4.2104	1.000	1	4.218
barbell	25	145	1	stoer-wagner	10	0.0094	1.000	1	0.011
barbell	25	145	1	karger	100	0.0056	0.990	1	0.006
barbell	25	145	1	karger-stein	100	4.2476	1.000	1	4.237
barbell	25	145	2	stoer-wagner	10	0.0086	1.000	1	0.009
barbell	25	145	2	karger	100	0.0055	0.980	2	0.011
barbell	25	145	2	karger-stein	100	4.2339	1.000	1	4.241
barbell	50	601	0	stoer-wagner	10	0.0705	1.000	1	0.072
barbell	50	601	0	karger	100	0.0238	1.000	1	0.023
barbell	50	601	0	karger-stein	100	17.3646	1.000	1	17.305
barbell	50	601	1	stoer-wagner	10	0.0698	1.000	1	0.070
barbell	50	601	1	karger	100	0.0240	1.000	1	0.024
barbell	50	601	1	karger-stein	100	17.4540	1.000	1	17.366
barbell	50	601	2	stoer-wagner	10	0.0670	1.000	1	0.068
barbell	50	601	2	karger	100	0.0235	1.000	1	0.023
barbell	50	601	2	karger-stein	100	17.4379	1.000	1	17.388
barbell	100	2451	0	stoer-wagner	10	0.6238	1.000	1	0.625
barbell	100	2451	0	karger	100	0.1095	1.000	1	0.109
barbell	100	2451	0	karger-stein	100	69.3958	1.000	1	69.461
barbell	100	2451	1	stoer-wagner	10	0.6319	1.000	1	0.636
barbell	100	2451	1	karger	100	0.1044	1.000	1	0.109
barbell	100	2451	1	karger-stein	100	69.6865	1.000	1	69.574
barbell	100	2451	2	stoer-wagner	10	0.6605	1.000	1	0.675
barbell	100	2451	2	karger	100	0.1072	1.000	1	0.110
barbell	100	2451	2	karger-stein	100	69.7742	1.000	1	69.658

cycle	25	25	0	stoer-wagner	10	0.0090	1.000	1	0.009
cycle	25	25	0	karger	100	0.0015	1.000	1	0.002
cycle	25	25	0	karger-stein	100	4.2026	1.000	1	4.200
cycle	25	25	1	stoer-wagner	10	0.0096	1.000	1	0.009
cycle	25	25	1	karger	100	0.0015	1.000	1	0.002
cycle	25	25	1	karger-stein	100	4.2975	1.000	1	4.288
cycle	25	25	2	stoer-wagner	10	0.0089	1.000	1	0.009
cycle	25	25	2	karger	100	0.0015	1.000	1	0.002
cycle	25	25	2	karger-stein	100	4.2528	1.000	1	4.241
cycle	50	50	0	stoer-wagner	10	0.0609	1.000	1	0.063
cycle	50	50	0	karger	100	0.0025	1.000	1	0.003
cycle	50	50	0	karger-stein	100	17.4221	1.000	1	17.392
cycle	50	50	1	stoer-wagner	10	0.0611	1.000	1	0.062
cycle	50	50	1	karger	100	0.0026	1.000	1	0.003
cycle	50	50	1	karger-stein	100	17.4577	1.000	1	17.426
cycle	50	50	2	stoer-wagner	10	0.0610	1.000	1	0.062
cycle	50	50	2	karger	100	0.0026	1.000	1	0.003
cycle	50	50	2	karger-stein	100	17.3662	1.000	1	17.345
cycle	100	100	0	stoer-wagner	10	0.4209	1.000	1	0.421
cycle	100	100	0	karger	100	0.0048	1.000	1	0.005
cycle	100	100	0	karger-stein	100	69.6441	1.000	1	69.632
cycle	100	100	1	stoer-wagner	10	0.4211	1.000	1	0.422
cycle	100	100	1	karger	100	0.0047	1.000	1	0.005
cycle	100	100	1	karger-stein	100	69.5619	1.000	1	69.561
cycle	100	100	2	stoer-wagner	10	0.4206	1.000	1	0.421
cycle	100	100	2	karger	100	0.0048	1.000	1	0.005
cycle	100	100	2	karger-stein	100	70.0271	1.000	1	69.833
many_cuts	25	95	0	stoer-wagner	10	0.0097	1.000	1	0.010
many_cuts	25	95	0	karger	100	0.0038	0.150	29	0.113
many_cuts	25	95	0	karger-stein	100	4.6675	0.980	2	9.316
many_cuts	25	95	1	stoer-wagner	10	0.0095	1.000	1	0.010
many_cuts	25	95	1	karger	100	0.0038	0.150	29	0.109
many_cuts	25	95	1	karger-stein	100	4.7028	0.980	2	9.363
many_cuts	25	95	2	stoer-wagner	10	0.0096	1.000	1	0.011
many_cuts	25	95	2	karger	100	0.0037	0.150	29	0.109
many_cuts	25	95	2	karger-stein	100	4.6931	0.980	2	9.361
many_cuts	50	161	0	stoer-wagner	10	0.0647	1.000	1	0.067
many_cuts	50	161	0	karger	100	0.0066	0.140	31	0.206
many_cuts	50	161	0	karger-stein	100	17.9203	0.970	2	35.785
many_cuts	50	161	1	stoer-wagner	10	0.0665	1.000	1	0.068
many_cuts	50	161	1	karger	100	0.0066	0.140	31	0.205
many_cuts	50	161	1	karger-stein	100	17.8894	0.970	2	35.760
many_cuts	50	161	2	stoer-wagner	10	0.0645	1.000	1	0.065
many_cuts	50	161	2	karger	100	0.0065	0.140	31	0.204
many_cuts	50	161	2	karger-stein	100	17.9128	0.970	2	35.723
many_cuts	100	322	0	stoer-wagner	10	0.4767	1.000	1	0.477
many_cuts	100	322	0	karger	100	0.0136	0.100	44	0.616
many_cuts	100	322	0	karger-stein	100	69.4062	0.970	2	138.899
many_cuts	100	322	1	stoer-wagner	10	0.4743	1.000	1	0.475
many_cuts	100	322	1	karger	100	0.0130	0.100	44	0.626
many_cuts	100	322	1	karger-stein	100	69.3649	0.970	2	138.517
many_cuts	100	322	2	stoer-wagner	10	0.4719	1.000	1	0.473
many_cuts	100	322	2	karger	100	0.0136	0.100	44	0.605
many_cuts	100	322	2	karger-stein	100	69.1422	0.970	2	138.441

The overall answer to our questions is that for the ranges of problem sizes we investigated, Stoer-Wagner performed better when the probability that a single random contraction would be unsuccessful was high; Karger performed better when the chance of success with a single trial was high; and Karger-Stein lost by two to three orders of magnitude on every family of graphs. None of the results we observed were forecasted solely by asymptotic analyses. Instead, each result came down to what the algorithms did during a single run, and how frequently such runs

succeeded. We will look at each of these two concepts individually before looking at their combination.

What One Run Costs

Figure 1(a) shows the median time of one run at n = 100, and Figure 1(b) shows how much that time grows when n doubles from 50 to 100: a factor of 2 means time proportional to n, 4 means n^2, and 8 means n^3. The three algorithms separate cleanly, in a way that matches how each one stores the graph.

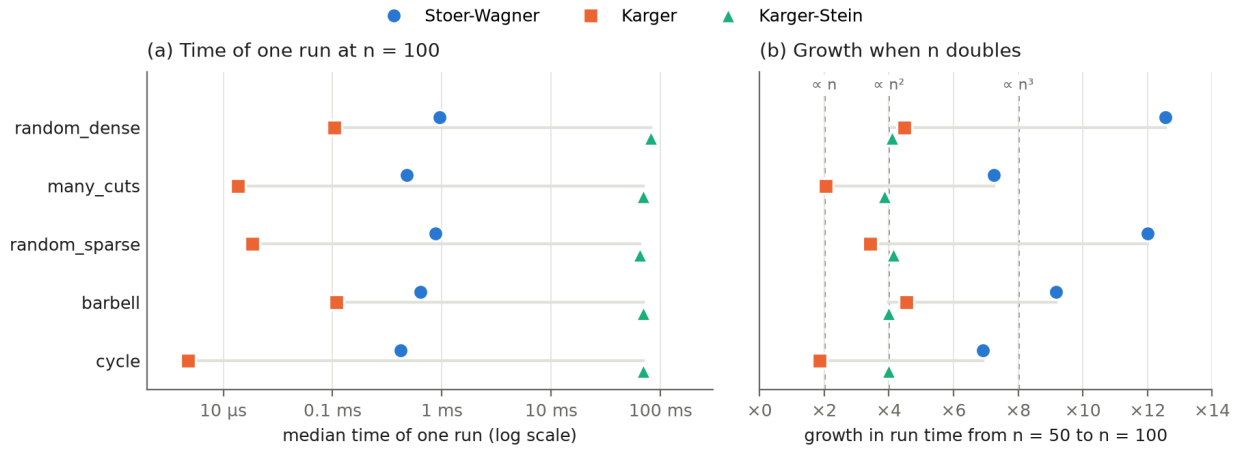


Figure 1. (a) Median time of one run at n = 100, on a log scale. (b) Growth in that time from n = 50 to n = 100. Dashed lines mark the growth expected for a time proportional to n, n^2 and n^3.
Table 2. Median time of one run at n = 100, and the factor by which it grows when n doubles (25 to 50, then 50 to 100).

Family	Edges	Stoer-Wagner	Karger	Karger-Stein
random_dense	2,461	0.95 ms (×8.1, ×12.6)	0.103 ms (×4.4, ×4.5)	81.1 ms (×4.1, ×4.1)
random_sparse	485	0.87 ms (×7.5, ×12.0)	0.019 ms (×3.3, ×3.4)	64.6 ms (×4.0, ×4.1)
barbell	2,451	0.63 ms (×7.2, ×9.2)	0.108 ms (×4.3, ×4.5)	69.7 ms (×4.1, ×4.0)
cycle	100	0.42 ms (×6.7, ×6.9)	0.005 ms (×1.7, ×1.9)	69.7 ms (×4.1, ×4.0)
many_cuts	322	0.48 ms (×6.8, ×7.3)	0.014 ms (×1.8, ×2.1)	69.4 ms (×3.8, ×3.9)

Stoer-Wagner's cost is set by n. It grows ×6.7 to ×12.6 per doubling, close to $\Theta(n^3)$, because its matrix makes it execute the same loop iterations on a 100-edge cycle as on a 2,461-edge dense graph. Yet the dense graph takes 2.2 times as long, which we explain at the end of this section.
Karger's cost is set by m. Its time groups by edge count, not by family: about 40 ns per edge in every family at n = 100, the cost of drawing and sorting a random key. On cycle and many_cuts, whose edge counts grow linearly, it grows only ×2 per doubling. Karger is the only algorithm whose cost depends on m, and that decides several comparisons below.
Karger-Stein's cost is set by the size of its recursion tree. It grows ×4 per doubling in every

family, and the cycle costs 86% as much as the dense graph with one twenty-fifth of the edges, which the textbook O(n² log n) analysis does not predict. The recursion explains it. With t = ⌈1 + n/√2⌉, graphs of 10 or fewer vertices lose one vertex per level but still branch twice, so a run makes exactly 13.1n² calls (131,071 at n = 100). Each call costs a near-constant 0.47 to 0.62 µs, mostly spent allocating structures for a graph of a handful of vertices. One Karger-Stein run therefore costs as much as 640 to 14,500 Karger runs.

Figure 2 shows how often one run succeeds. Every rate is far above its bound, by 45 to 4,950 times for Karger, and the size of the gap reflects each family's cut structure.

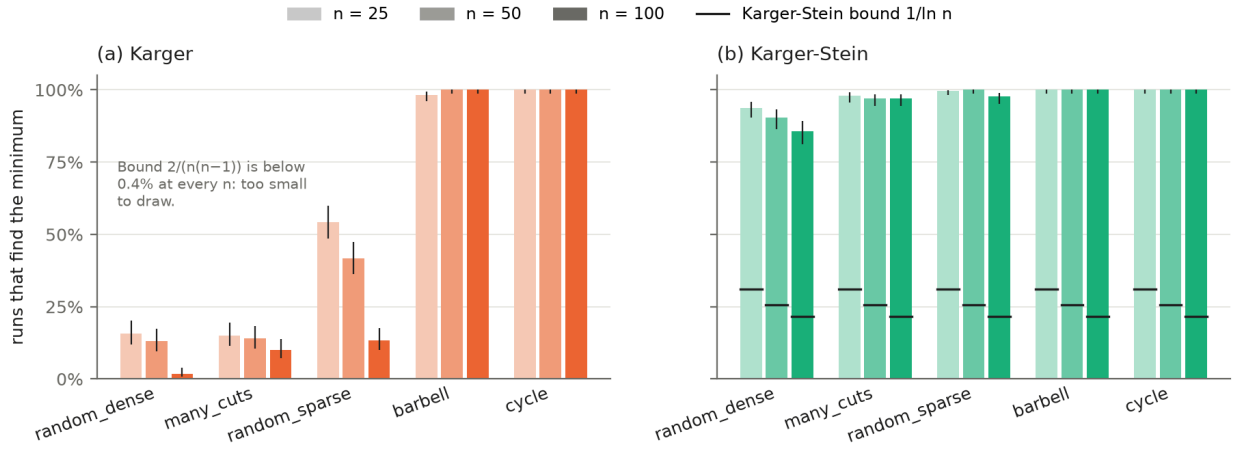


Figure 2. Share of runs that find the minimum cut, by family and n (300 runs per bar; thin black lines are 95% Wilson intervals). Black ticks in (b) mark Karger-Stein's bound 1/ln n. Karger's bound is too small to draw.

Karger performs well on both the cycle and barbell. A contraction of a cycle results in a cycle. Thus, the final two vertices will always be connected by exactly two edges, which is always a minimum cut. The bound counts only one of the n(n-1)/2 minimum cuts of a cycle.
The barbell, however, contains a single "bridge" with a weight of 1, and since the average weight of a clique edge is 50, the bridge will rarely be selected for contraction.

In random dense graphs, Karger performs worse than for any other class of graphs (16%, 13%, then 1.7% at n=100). However, these random dense graphs contain no pre-determined or "planted" structures; their only structure is that they are the result of selecting edges randomly. In such graphs, the minimum cut is simply the lightest vertex. As n increases, the vertex weights become more concentrated around their mean, because their spread grows only like √n while their mean increases linearly (n). This results in multiple vertices having approximately the same weight. With so many similar weights, random dense graphs become "many-near-minimum-cuts" cases by themselves. Sparse graphs have more variation in degree than dense graphs. Karger is successful more often than on dense graphs (13% at n = 100) but exhibits great variability between instances (14%, 26%, and 85% at n=50); success in sparse graphs depends upon how much smaller the lightest cut is compared to the second smallest cut rather than what type of graph it was.

In many_cuts Karger successfully recovers a minimum cut 10-15% of the time as designed. Its failures end on cuts of weight 6. Success rates fall only gradually, probably because the total number of minimum cuts grows with the number of cliques.

Finally, Karger-Stein achieves success rates ranging from 86-100% on a run-by-run basis, three to four-and-a-half times its bound. It does this by repeating only the contractions that occur

toward the later stages of contraction; it is here that most of the risk lies. Near the base of the recursion tree, this implementation branches at every single-vertex contraction, which raises success even further. This same branching explains why a run costs exactly 13.1n² calls; increased success and increased cost are caused by a single large recursion tree.

Time To 99% Confidence

Combining the two gives the time to 99% confidence (Figure 3 and Table 2).

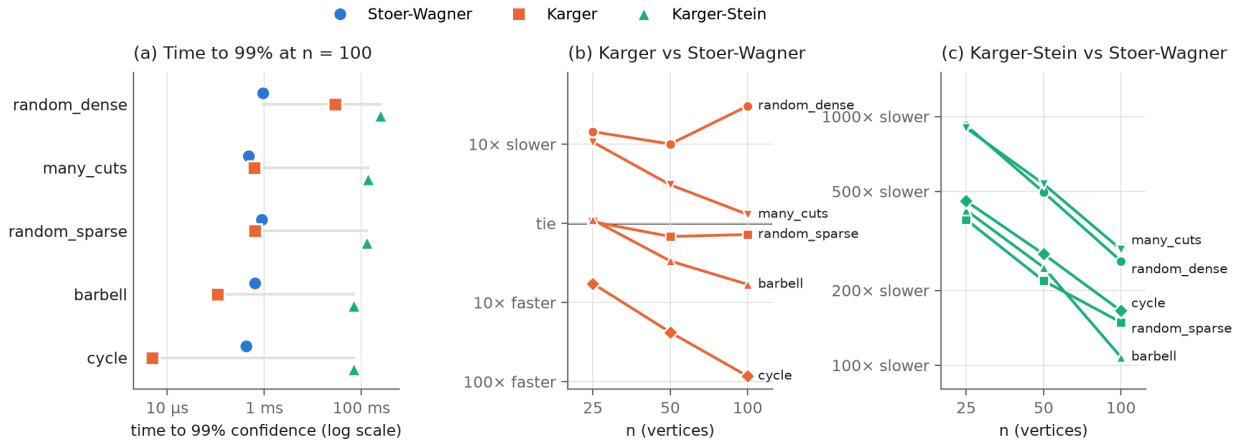


Figure 3. (a) Time to find the minimum with 99% confidence at n = 100 (k runs × mean run time; Stoer-Wagner needs one run). (b) and (c) The same time for Karger and Karger-Stein divided by Stoer-Wagner's, against n. Below the tie line in (b), Karger is faster.

Family	Stoer-Wagner	Karger	Karger-Stein	Fastest
random_dense	0.94 ms	28.6 ms (275)	247 ms (3)	Stoer-Wagner, 30× ahead of Karger
random_sparse	0.87 ms	0.63 ms (33)	130 ms (2)	Karger, 1.4× ahead
barbell	0.65 ms	0.11 ms (1)	69.6 ms (1)	Karger, 6× ahead
cycle	0.42 ms	0.005 ms (1)	69.7 ms (1)	Karger, 85× ahead
many_cuts	0.48 ms	0.62 ms (44)	139 ms (2)	Stoer-Wagner, 1.3× ahead

The reason for this performance difference stems primarily from the type of graph being used. Karger performs best when a single run usually succeeds and there are few edges. As shown in Table 3, a single 5 microsecond run is enough on the cycle, while Stoer-Wagner runs 99 phases across its entire matrix. Using the same logic, on random_sparse Karger needs 33 inexpensive runs, which together still outperform a single calculation that requires O(n^3) time.
This trend reverses once the number of runs needed increases substantially. When the contraction process becomes unreliable, Stoer-Wagner has an advantage. For example, on random_dense, Karger goes from needing 34 runs at n=50 to needing 275 runs at n=100, an 8-fold increase in the number of runs. In addition, each individual run now costs 4.5x as much, thus increasing the overall time to 99% by 36 times.

In comparison, Stoer-Wagner's time to reach 99% grew only 12 times during this period. There may also be some value in examining why these two algorithms have performed differently. For instance, looking at many_cuts (figure 3b), Stoer-Wagner leads Karger by approximately 10.6x

at n=25. However, this lead decreases as n increases to 1.3x at n=100. Given that many_cuts was designed to defeat Karger, we can infer that if current trends continue then Karger will likely surpass Stoer-Wagner before reaching a vertex count of approximately 200.

As previously mentioned, Karger-Stein's loss compared to Stoer-Wagner does decrease with each doubling (figure 3c), by about 1.9x, close to the n/log n ratio predicted by their respective bounds. However, based on current rates of improvement, it appears that Karger-Stein will need to reach approximately 40,000 to 50,000 vertices before drawing level with Stoer-Wagner. At that size, Stoer-Wagner's matrix alone would require 13 to 20 gigabytes of memory.

Why does Stoer-Wagner's speed depend upon the graph?
Although Stoer-Wagner's implementation includes identical code sequences for each graph of a particular size, there exists a measurable time difference when comparing its execution on the cycle versus random_dense. Specifically, at n=100, Stoer-Wagner executed in 0.42 milliseconds on the cycle while taking 0.95 milliseconds to execute on random_dense.

Our best explanation for this discrepancy is branch prediction. Since Stoer-Wagner tests inA[v] in index order, a processor runs that loop faster when the outcome of the test follows a predictable pattern. The cycle has a simple pattern, where maximum adjacency ordering causes A to follow a continuous block of indices, many_cuts fills a clique at a time, and random graphs scatter A throughout.

To measure the frequency at which inA changes value during the scan (analysis/sw_probe.cpp provides the actual data for these measurements), Figure 4 illustrates how time increases in proportion to the number of times inA changes value (a correlation coefficient of 0.98 and approximately 4.4 nanoseconds per change, roughly equivalent to the cost of one branch-prediction miss).

At first glance, this seems like a fairly solid correlation; however, simply measuring the branch-missing counter or timing a branch-less version of the loop would provide conclusive evidence.

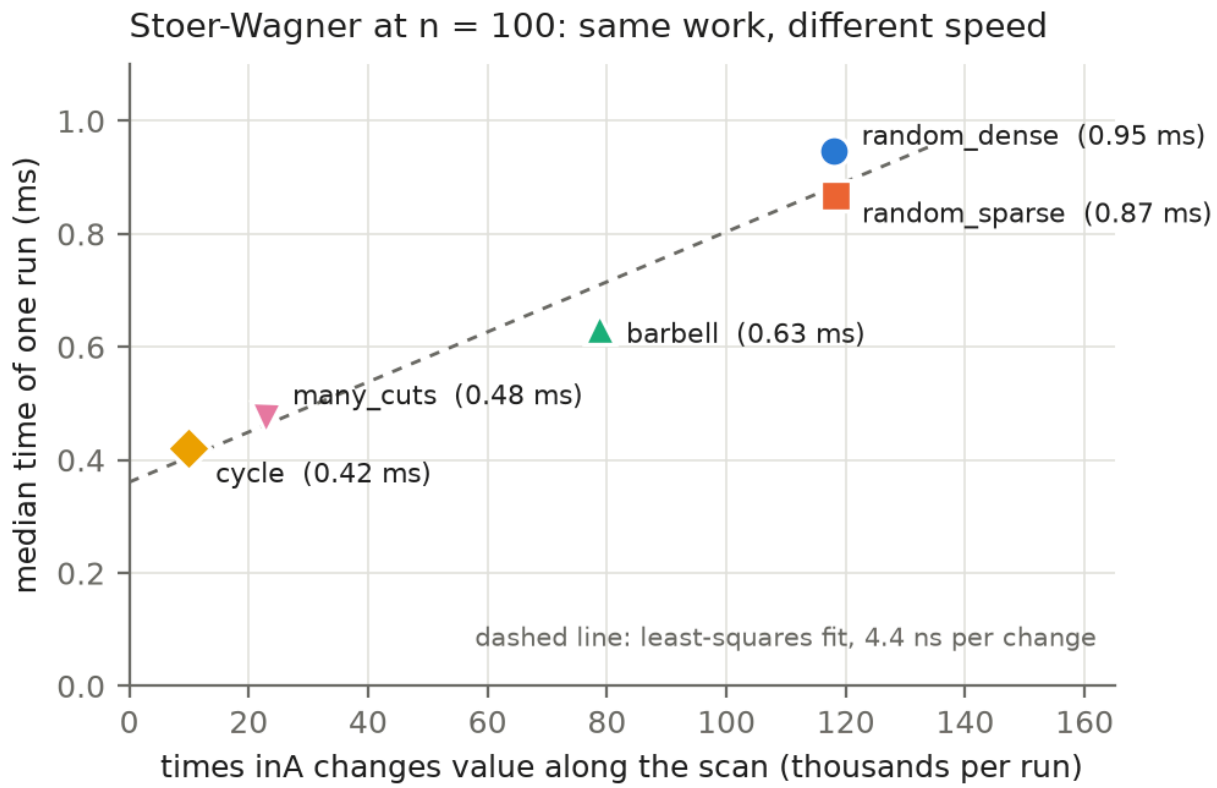


Figure 4. Stoer-Wagner's median time at n = 100 against the number of times inA changes value along its scans, averaged over the three benchmark instances of each family.

How the results compare with our predictions

Table 3 compares the predictions we committed before running anything with the results.
Table 3. Predictions and outcomes.

Prediction	What we measured	Outcome
Dense graphs: Stoer-Wagner first, Karger-Stein second, Karger last (Kartikay)	Stoer-Wagner first, but Karger beats Karger-Stein by 9× at n = 100	Partly held
Dense graphs: Karger-Stein fastest (Brian, Ethan)	Karger-Stein slowest on every family, 108 to 920× behind Stoer-Wagner	Did not hold
Sparse graphs: Stoer-Wagner first, Karger last (Kartikay, Brian, Ethan)	Karger fastest at n = 50 and 100; Karger-Stein last	Did not hold
Cycles and planted cuts: Karger beats Stoer-Wagner at every size (Kartikay)	Karger ahead on cycle (up to 85×) and barbell (up to 6×)	Held
Karger-Stein overtakes Stoer-Wagner only above about 1,000 vertices (Kartikay, Brian; Ethan predicted about 500)	No crossover up to 100; the trend puts it near 40,000 to 50,000	Partly held

Prediction	What we measured	Outcome
Karger's success 100× or more above its bound on random graphs, several percent at n = 100 (Kartikay)	Sparse: 660×, 13%. Dense: 82×, 1.7%	Partly held
Hardest family many near-minimum cuts, easiest the cycle (Kartikay)	Cycle 100%; many_cuts 10 to 15%; random_dense lower still at n = 100	Held
Cycles are the hardest for the randomised algorithms (Ethan)	Cycles are the easiest: every run succeeds	Did not hold
Karger-Stein succeeds more often than Karger (Brian)	86 to 100% against 1.7 to 100%	Held
Stoer-Wagner grows ×8 per doubling, with no dependence on structure (Kartikay)	×6.7 to ×12.6; 2.2× slower on random_dense than on the cycle at n = 100	Partly held

The predictions that failed share one assumption: that asymptotic bounds would decide the comparison at the sizes we could test. What decided it was the cost of one concrete operation in each implementation, and how often one random run succeeds on a given cut structure. The next section takes up the surprises this produced.

5. What Surprised Us

I was very surprised to see specifically for both the barbell and many cuts input family results. Averaging out the benchmark results for all runs and for all *n* size, the results in mean ms average and success rate is as follows:
Barbell:
Karger: ~0.046ms & 99.44%
Karger-Stein: ~30.383ms & 100.00%
Stoer-Wagner: ~0.242ms & 100.00%
Cycle:
Karger: ~0.003ms & 100.00%
Karger-Stein: ~30.435ms & 100.00%
Stoer-Wagner: ~0.164ms & 100.00%

This result is shocking as the Karger-Stein averages ~30ms slower on average than both Karger and Stoer-Wagner in both input families in this example. The general consensus in the predictions was that in terms of speed, Karger-Stein would be faster than Karger but the results

for these 2 input families specifically show otherwise. This is because Karger-Stein is supposedly an optimised extension of Karger as it uses divide and conquer whilst Karger uses random contraction. The difference in speed was most likely due to Karger-Stein using a recursive contraction tree that branches multiple times. This ideally should be superior to what Karger uses but it raises an overhead issue where Karger-Stein gets penalised on smaller graphs as seen in the results. Furthermore, the success rate difference is miniscule in the barbell and the exact same for cycle. It is also surprising to see that the Stoer-Wagner algorithm kept up with Karger in terms of efficiency and is way faster than Karger-Stein. It is a deterministic algorithm yet it places second in terms of efficiency.

Looking at the random dense, sparse and many cuts input family, the results are:
Random Dense:
Karger: ~0.044ms & 10.11%
Karger-Stein: ~35.660ms & 89.89%
Stoer-Wagner: ~0.344ms & 100.00%
Random Sparse:
Karger: ~0.009ms & 36.44%
Karger-Stein: ~28.194ms & 99.11%
Stoer-Wagner: ~0.318ms & 100.00%
Many Cuts:
Karger: ~0.008ms & 13.00%
Karger-Stein: ~30.620ms & 97.33%
Stoer-Wagner: ~0.184ms & 100.00%

Kartikay:
What surprised me here was the success rate of Karger specifically in random dense and many cuts. It was widely known that Karger was bound to suffer in terms of success rate in comparison to the deterministic approach and the more “optimised” Karger-Stein algorithm but the difference here was surprising. If we take a look at Karger’s success rate in random dense $n = 100$ (refer to the benchmark results table in section 4), the success rate is 1.7%. That is absurdly low as some of our predictions were Karger’s success rate would not drop that much. This is due to the random dense and sparse graphs having more edge cuts which obscures the minimum cut and as a result, Karger’s random choices becomes a double edge sword giving it a higher chance of pointing towards a suboptimal cut. This is also why to obtain a 99% success rate, it must run $O(n^2 \log n)$ times.

Another surprise referring to the averages of all 5 input families is the fact that Karger was almost perfect in terms of success rate on barbell and cycle family but was inaccurate on single runs in random dense, sparse and many cuts. The difference between them is

Before running anything, I predicted that Stoer-Wagner's running time would depend only on n, because my adjacency-matrix implementation performs exactly the same loop iterations on every graph of a given size. The growth rate matched, at about ×7 to ×12 per doubling, close to n^3. The surprise was the constant. At n = 100, Stoer-Wagner took 0.42 ms on the cycle and 0.95 ms on random_dense, 2.2 times as long for identical work, with many_cuts (0.48 ms) and

barbell (0.63 ms) in between.

Since the work was identical, the difference had to come from how fast the processor executed it. My best explanation is branch prediction. Each scan tests inA[v] in index order. On a cycle, maximum adjacency ordering walks around the cycle, so A is one contiguous block of indices and the test is easy to predict; on random graphs A is scattered, so the processor mispredicts far more often. Counting how often inA changes value on the exact benchmark graphs gave an almost straight line against time (Figure 4: correlation 0.98, about 4.4 ns per change, close to the cost of one mispredicted branch). It also explains why the growth per doubling was ×12 on the random graphs but only ×7 on the cycle.

This changed how I think about complexity: two inputs with the same operation count can still differ by more than 2× in time. It remains a correlation, so reading the hardware branch-miss counters, or timing a branch-free version of the loop, would confirm it.

6. What we'd do Next

What's the natural extension of this project?
The natural extension of this project is testing more algorithms, through optimising them to see if the benchmarks change. Karger-Stein can be improved by using parallel cuts to compute the branches at the same time instead of separately. This would reduce Karger-Steins time, which would potentially allow for it to beat Stoer-Wagner at small graph sizes and make the crossover point earlier be earlier in vertices. A hybrid Stoer-Wagnet and Karger approach could also be done, where Karger is used to contract a large graph down into supernodes first, and then using Stoer-Wagner in the remainder. This could have the benefits of Kargers fast scaling, while also not having to repeat the randomised contraction on small graphs where Stoer-Wagner is faster.

An extension could also include flow-based algorithms such as Ford-Fulkerson or Hao-Orlin, or other deterministic algorithms like Nagamochi-Ibaraki, by implementing them and testing them against the benchmarks. This could allow for the benchmarking to be more complete, and see if other deterministic ones stack up to Stoer-Wagner, and how they all compare to flow-based minimum-cut algorithms.

What questions did the work open up that you didn't have time to answer?
What is the cost of randomising, and are there any other more efficient ways to generate random numbers? Both Karger and Karger-Stein call std::mt19937_64 and then normalises the weights. If instead of the standard library random number generator, a more lightweight one was used, would Karger and Karger-Steins execution times drop and how would it change the crossover point.

Where the exact density crossover point is on each of the graphs? On sparse graphs Stoer-Wagner wins and on dense graphs Karger-Stein eventually wins, but the exact threshold was never plotted, and was only estimated.

Can memory allocation be eliminated from Karger-Stein and Stoer-Wagner? To achieve theoretical time bounds, the Karger-Stein implementation builds a new compact Graph at every recursive step, which allocates new vectors. The Stoer-Wagner allocates priority queues and distance arrays. We did not measure the penalties of these heap allocations, and if they were rewritten with this in mind, which one would benefit the most.

7. Individual Contributions

Kartikay: Beyond Stoer-Wagner (stoer_wagner/), I coordinated the project. I proposed the topic and the split of algorithms, set the week plan, and set up the shared repository: the graph interface, Makefile, test runner and README. I reviewed and merged everyone's pull requests and checked that each member's code worked and matched its purpose. That checking caught Karger being switched off in the harness, the many_cuts generator falling back to a cycle, and Karger-Stein's early slowness, which led to Ethan's speed fix. I also built the integration layer: the benchmark harness, Karger's registration in it, and the fix that made many_cuts a real test. Using the results, I produced the analysis scripts and figures and wrote Sections 3 and 4, the Stoer-Wagner part of Section 1, and my part of Section 5. Finally, I made the video slides and fixed the factual errors in the assembled report.

Brian: I worked on the Karger implementation as seen on the karger and karger-old-version branch. I was also responsible for implementing/creating the brute force checker in common/brute_force.cpp and hpp. I did Karger in Section 1 and section 5. I also produced the benchmarks results table.

Ethan
I worked on the Karger-Stein implementation, created the generator for the random, barbell, cycle and many cuts graph. I did the Karger-Stein and Problem in Section 1 and Section 6

8. AI Use

Refer to logs/ai-use.md in the main branch.

Kartikay used Claude (Claude Code) throughout the project; The full record is in logs/ai-use.md. Kartikay used Claude for four main things: drafting the initial repository (the shared graph interface, Makefile, test runner with hand-checked cases, and README); checking that each member's code worked and did what it was meant to do; checking that the group's results were computed correctly; and organising the results data into tables and figures. Claude also drafted the benchmark harness, the analysis scripts and figures. Every one of these was reviewed, and several were corrected, as described below.

Brian used ChatGPT to help implement the brute-force checker in brute_force.cpp and brute_force.hpp. Help in implementing Karger's algorithm in karger.cpp.Brian used Gemini to

take the benchmark results and find the averages of each instance.

Where the AI was wrong, unhelpful or misleading

1. Wrong numbers in a theory table. To help us write predictions, Claude produced a table of costs. It said Karger needs about $2.3n^2/2$ runs for 99% confidence and that Karger-Stein's total cost is $O(n^2 \log^3 n)$. Kartikay's own answers said $2.3n^2$ and $O(n^2 \log^2 n)$, and the mismatch exposed the error: the first was off by a factor of two, and the second was the bound for success "with high probability", not for a fixed 99%. We rederived both and corrected the table. The lesson was that a neat table looks authoritative whether or not it is right.

2. Convincing analysis with wrong details. Claude's first draft of the analysis read well, but checking each number against results/trial.csv found errors: Karger-Stein's cost per recursive call was given as 0.52 to 0.62 µs instead of 0.47 to 0.62 µs, the size below which its recursion removes one vertex per level was given as 6 instead of 10, and the first branch-counting probe used different random graphs from the benchmark. All were fixed before the draft was used. Later, a reworded version of the analysis carried about ten factual errors, such as "many_cuts was designed to favour Karger" (it was designed to defeat it) and "Karger performs better on random dense graphs" (it performs worse). A full check of the report against the data caught them.

3. Output that needed reworking. The first figures plotted fifteen overlapping lines on log-log axes and used the same colours for algorithms in one panel and for input families in another. The first draft of Sections 3 and 4 was about 3,800 words. Kartikay pushed back on both; the figures were redrawn as dot and bar charts with one colour per algorithm, and the sections were cut to about 2,350 words.

AI in group work

Where it helped the group function. The biggest benefit was checking each other's work quickly and concretely. Using Claude, Kartikay confirmed that Brian's rewritten Karger performs a single run with union-find and scales as expected; found that Karger had not been switched on in the harness, so the first full run would have produced no Karger data; found that Ethan's many_cuts family silently became a cycle whenever n was not a multiple of four and was too easy for Karger to test anything; timed Ethan's first Karger-Stein at about 20 seconds per run at $n = 200$, which led to his speed fix; and caught a merge that had credited Ethan's generator code to the AI. Drafting the repository skeleton early gave all three of us a shared interface and test runner before anyone wrote an algorithm, which made integration straightforward. Turning 9,450 rows of results into a few tables and figures saved time that went into explaining the results instead.

Where it hurt, or threatened to. Polished prose was a risk. Analysis written fluently about Brian's and Ethan's algorithms is only useful if they can explain it themselves in the video, so each member needed to check the paragraphs about their own algorithm. Finally, AI output always needed checking number by number: almost every error above looked plausible until it was compared with the data or the code.